CUDA-sDBSCAN: GPU Accelerated Density Based Clustering With Random Projections

Hugo Phibbs and Ninh Pham

***Abstract*—**

I. Introduction

DBSCAN [2] is density based clustering based algorithm with many applications in areas such as machine learning, data mining and biology [**?**]. It's applicability has stood the test of time - appearing in a plethora of data analysis tools such as ELKI, scikit-learn, R and Weka.

II. Related Work

There have been a myriad of research in the past to adapt DBSCAN for large scale datasets. These attempts can be broadly classified into the usage of data indexing, approximating clusters, specialised hardware and distributed computing.

Work to scale up DBSCAN with data indexing [**?**], [**?**]

why are these refs not showing up?

can be effective at increasing the scalability of DBSCAN

reference for this

. Although they have shown to be successful for low dimensional datasets, they are ill-suited to large scale applications because of run-off effects with the complexity of effectively indexing high dimensional data.

Research to approximate DBSCAN [3], [7], [8] has also proved effective at scaling DBSCAN. Sampling approaches such as K-DBSCAN [**?**] and sngDBSCAN [3] are potentially efficient, except they can rely on the a wise choice of sampling parameters - which can be difficult.

feel like more can be said here

Due to the parallelizable nature of DBSCAN, other approaches [**?**], [**?**], [**?**] have focused on using a GPU to accelerate DBSCAN. CUDA-DClust+ [**?**] utilizes a tree indexing structure as-well as a GPU to accelerate computation. While it is excellent for applications with small scale data-sets, it is limited to this use case due to complexity of creating an indexing tree over large dimensions. Andrade et al's G-DBSCAN [**?**] focuses on using graph principles along with a GPU to scale DBSCAN. However, due to it's usage of an graph adjacency list, G-DBSCAN can prove to be severely memory intensive [**?**]. There is also little experimental data to support the usage of G-DBSCAN dimensions greater than 2.

honestly, not too sure about this! - its a claim made in CUDA-DClust+ paper - end of p40

III. Preliminaries

A. DBSCAN

Perhaps comment that the below is a naive implementation of DBSCAN?

In a nutshell, DBSCAN [1] is an algorithm to cluster related data points together based on a given distance metric. The algorithm is composed of two key processes: the identification of core points and the formation of clusters.

This is a little premature to mention 'core points' when they haven't even been defined yet

For the sake of explanation consider a set of data points $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, where $dim(\mathbf{x}_i) = d$. The identification of core points can be completed as follows:

1) For each data point $q \in \mathbf{X}$ find the distance $dist(\mathbf{x}, \mathbf{q})$ to all other data points $\mathbf{x} \in \mathbf{X}$. If $dist(\mathbf{x}, \mathbf{q})$ is less than the maximum clustering distance ϵ, add $\mathbf{x}$ to the neighbourhood set of $\mathbf{q}$, denoted by $B_\epsilon(\mathbf{q})$
2) For each $\mathbf{q} \in \mathbf{X}$, if $|B_\epsilon(\mathbf{q})|$ is greater than or equal to a threshold $minPts$, the point $\mathbf{q}$ is a *core point*. Add the tuple $(\mathbf{q}, B_\epsilon(\mathbf{q}))$ to the core point set C.

The pseudo code for the identification of key points is shown in algorithm 1.

Algorithm 1 DBSCAN, classification of core points

Input: Data points $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, ϵ, $minPts$
Output: Set containing core points and their ϵ-neighbourhoods
Initialize $C = \emptyset$
for each data point $\mathbf{q} \in \mathbf{X}$ **do**
    Initialize $B_\epsilon(\mathbf{q}) = \emptyset$
    for each data point $\mathbf{x} \in \mathbf{X}$ **do**
        Compute $dist(\mathbf{x}, \mathbf{q})$
        if $dist(\mathbf{x}, \mathbf{q}) < \epsilon$ **then**
            Add $\mathbf{x}$ to $B_\epsilon(\mathbf{q})$
        end if
    end for
    if $|B_\epsilon(\mathbf{q})| \geq minPts$ **then**
        Add $(\mathbf{q}, B_\epsilon(\mathbf{q}))$ to C
    end if
end for

Once core points are identified, the data set can be partitioned into clusters. A cluster is formed by considering all points that are within *direct* or *transitive* ϵ distance from each other. Given the identification of core points and their

neighbourhoods, clusters can be formed using algorithm 2. The result of the DBSCAN algorithm is a graph G, where each connected component is a cluster of the data set $\mathbf{X}$

Algorithm 2 DBSCAN, forming clusters

Inputs: $\mathbf{X}, \epsilon, minPts$ the set $C = \{(\mathbf{q}, B_\epsilon(\mathbf{q})) \,|\, \mathbf{q} \text{ is core}\}$
$G \leftarrow$ initialize empty graph
for each $\mathbf{q} \in C$ **do**
    Add an edge (and possibly a vertex or vertices) in G from $\mathbf{q}$ to all *core* points in $B_\epsilon(\mathbf{q})$
    Add an edge (and possibly a vertex) in G from $\mathbf{q}$ to *non-core* points $\mathbf{x} \in B_\epsilon(\mathbf{q})$ if $\mathbf{x}$ is not connected
end for
return connected components of G

B. sDBSCAN

Comment here about the scalability of DBSCAN, and how this is addressed with sDBSCAN.

sDBSCAN [8] is an implementation of DBSCAN based on using random projections to rapidly speed up the ϵ-neighbourhood search for each particular data point.

Besides the given DBSCAN inputs of $\mathbf{X}$, ϵ, and $minPts$, sDBSCAN adds a few more parameters - these are described in table I.

Parameter	Description
$\mathbf{Y}$	A set of D random vectors with the same dimension as points in $\mathbf{X}$. Each random vector has coordinates sampled from the standard normal distribution. Formally: $\mathbf{Y} = \{\mathbf{r}^i \mid \mathbf{r}^i \in \mathbb{R}^d,\ \mathbf{r}^i \sim \mathcal{N}(0,1),\ i \in [D]\}$
k	The number of closest/furthest random vectors from $\mathbf{Y}$ to store for each query point $\mathbf{q} \in \mathbf{X}$
m	The number of closest/furthest query points from $\mathbf{X}$ to store for each random vector $\mathbf{r} \in \mathbf{Y}$

TABLE I
DEFINITIONS AND DESCRIPTIONS OF SDBSCAN'S PARAMETERS

The sDBSCAN algorithm can be summarised into a few key steps: pre-processing, identification of key points, and formation of clusters. The pre-processing steps are as follows:

1) For each random vector $\mathbf{r}_i \in \mathbf{Y}$, calculate it's projection to each query point $\mathbf{q} \in \mathbf{X}$.
2) For each query point $\mathbf{q} \in \mathbf{X}$, using the projections store its top k closest and furthest random vectors.
3) For each random vector $\mathbf{r}_i \in \mathbf{Y}$, using the projections, store its top m closest and furthest data set points.

From here, the preprocessing steps are essentially used to guide an informed search for the ϵ neighbourhood of each data point $\mathbf{x} \in \mathbf{X}$

1) Use the random projections to form an approximate ϵ-neighbourhood set $\widetilde{B}_\epsilon(\mathbf{q})$ for each data-set point $\mathbf{q}$.
2) Perform the standard DBSCAN clustering step 2 on the set of estimated core points and their neighbourhoods. $\widetilde{B}_\epsilon(\mathbf{q})$ sets for $\mathbf{q}$ identified as core.

The complete pseudocode is shown in algorithm 1.

Algorithm 3 Preprocessing

Inputs: $\mathbf{X} \subset S^{d-1}$, D random vectors $\mathbf{r}_i$, $k, m = O(minPts)$
for each $\mathbf{q} \in \mathbf{X}$, compute and store top-$k$ nearest and top-k farthest vectors $\mathbf{r}_i \in \mathbf{Y}$ to $\mathbf{q}$.
for each random vector $\mathbf{r}_i \in \mathbf{Y}$, compute and store top-$m$ nearest and top-m farthest points to $\mathbf{r}_i$.

Algorithm 4 Finding core points and their neighborhoods

Inputs: $\mathbf{X} \subset S^{d-1}$, $\mathbf{Y}$, $k, \epsilon, m = O(minPts)$
Initialize an empty set $\mathbf{r_q}$ for each $\mathbf{q} \in \mathbf{X}$
for each $\mathbf{q} \in \mathbf{X}$ **do**
    for each $\mathbf{r}_i$ from top-k nearest random vectors of $\mathbf{q}$ **do**
        for each $\mathbf{x}$ from top-m nearest points of $\mathbf{r}_i$ **do**
            if $dist(\mathbf{x}, \mathbf{q}) \leq \epsilon$ **then**
                Insert $\mathbf{x}$ into $\widetilde{B}_\epsilon(\mathbf{q})$ and insert $\mathbf{q}$ into $\widetilde{B}_\epsilon(\mathbf{x})$
            end if
        end for
    end for
    for each $\mathbf{r}_i$ from top-k farthest random vectors of $\mathbf{q}$ **do**
        for each $\mathbf{x}$ from top-m farthest points of $\mathbf{r}_i$ **do**
            if $dist(\mathbf{x}, \mathbf{q}) \leq \epsilon$ **then**
                Insert $\mathbf{x}$ into $\widetilde{B}_\epsilon(\mathbf{q})$ and insert $\mathbf{q}$ into $\widetilde{B}_\epsilon(\mathbf{x})$
            end if
        end for
    end for
end for
for each $\mathbf{q} \in \mathbf{X}$ **do**
    if $|\widetilde{B}_\epsilon(\mathbf{q})| \geq minPts$ **then**
        Identify $\mathbf{q}$ as the core point
        Store $\widetilde{B}_\epsilon(\mathbf{q})$ as an approximate result of $B_\epsilon(\mathbf{q})$
    end if
end for

C. GPU Programming with CUDA

A GPU (Graphical Processing Unit) is based around the idea of having using hundreds to thousands of independent cores to complete tasks in a parallelised manner. This strategy excels in areas such as linear algebra, sorting, and differential equations [6]. To ease the development of applications that utilize a GPU, NVIDIA, a major manufacturer of GPU chips, has created CUDA [4] to provide an API for GPU programming.

This sentence could be reworded

.

For the purposes of this paper, we will look at the typical architecture of a NVIDIA GPU, then see how this relates to the programming model of CUDA.

The CUDA programming model

can it be called a model?

abstracts the low levels of programming directly onto a GPU with a few key concepts. At the highest level there is a *kernel grid*, this executes a single set of instructions called a *kernel* across the GPU. A grid is sub components called *thread*

Algorithm 5 sDBSCAN

1: **Inputs**: $X \subset \mathcal{S}^{d-1}$, D random vectors $\mathbf{r}_i$, ε, $minPts$
2: Call Algorithm 3 for preprocessing with $m = O(minPts)$
3: Call Algorithm 4 to find the set
$C = \{(\mathbf{q}, \widetilde{B}_\varepsilon(\mathbf{q})) \mid \mathbf{q} \text{ is identified as core}\}$
4: Call DBSCAN (Alg. 1) given the output C from Algorithm 4

Fig. 1.

blocks these execute a number of individual kernel instructions - called *threads* - across one SM. A single *thread* is executed on one CUDA core - this is the lowest level of programming abstraction with CUDA. Figure 2 explains these concepts.

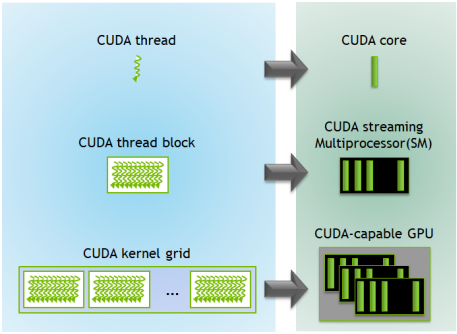


Fig. 2. Simplified overview of the CUDA programming model [5]

Threads are executed in groups called *warps*. To maximise occupancy of an SM while a block is being executed, the size of a block should be a multiple of 32

find a reference for this

.

IV. CUDA-sDBSCAN

V. Experiment

VI. Conclusion

References

[1] Ricardo Campello, Peer Kröger, Joerg Sander, and Arthur Zimek. Density-based clustering. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 10, 10 2019.
[2] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, KDD'96, page 226–231. AAAI Press, 1996.
[3] Heinrich Jiang, Jennifer Jang, and Jakub Łacki. Faster DBSCAN via subsampled similarity queries. _eprint: 2006.06743.
[4] NVIDIA. Cuda toolkit, 2023.
[5] NVIDIA Developer. Cuda refresher: The cuda programming model, 2023. Accessed: 2024-06-19.
[6] John D. Owens, Mike Houston, David Luebke, Simon Green, John E. Stone, and James C. Phillips. Gpu computing. *Proceedings of the IEEE*, 96(5):879–899, 2008.
[7] Johannes Schneider and Michalis Vlachos. Scalable density-based clustering with quality guarantees using random projections. 31.
[8] Haochuan Xu and Ninh Pham. Scalable density-based clustering with random projections, 2024.

‘